

Ministry of Higher Education

Soran University

Part 4 - Git Project Milestone

Second-hand Item Exchange System

GROUP MEMBERS

Lizaa Chato

Bushra Faris

Passar Khdir

Ariyan Walid

SUPERVISOR

Dr. Sirwan

1. Repository Setup

- **Platform:** GitHub (team-approved Git hosting platform).
- **Repository name:** second-hand-item-exchange-system
- **Repository URL:** _____ *(paste the GitHub link here after pushing)*
- **Access:** the supervisor, Dr. Sirwan, is added as a collaborator (or the repository is public), so the instructor can open it.
- **Branches:** `main` (reviewed work) and `docs/team-collaboration-record` (task branch, pull request).

2. README.md and .gitignore

README.md contains the project title, a short description, the team members, the current project status, the repository structure, and a link to the team workflow. No private personal data is included (names only, no emails or phone numbers).

.gitignore keeps secrets (`.env`, keys, tokens), editor and system files, Office temporary files, dependencies, build output, logs, local databases, and uploaded item images out of Git. A scan of the repository found no passwords, tokens or private keys.

```
# ---- Secrets and private data (NEVER commit these) ----
.env  .env.*  *.pem  *.key  *.p12  secrets.*  credentials.*  *.token
# ---- Operating system / editor ----
.DS_Store  Thumbs.db  .vscode/  .idea/  ~$*  *.tmp
# ---- Dependencies, build output, logs, databases ----
node_modules/  vendor/  dist/  build/  __pycache__/  *.log  *.sqlite
# ---- User-uploaded item images (stored on the server) ----
uploads/
```

3. Folder Organization

The existing Part 1-3 artefacts were added to clear folders. Nothing from Part 3 was recreated; the same documents and diagram images were version-controlled.

```
second-hand-item-exchange-system/
|-- README.md
|-- .gitignore
|-- docs/
|   |-- proposal/      Part1_Project_Proposal.pdf
|   |-- requirements/  Part2_SRS.pdf
|   |-- modeling/      Part3_System_Modeling.pdf
|   `-- team/          collaboration-record.md  (task branch)
|-- diagrams/          use-case, class, activity, sequence, ER (PNG)
|-- src/               README.md  (code to be added in implementation)
`-- tests/             README.md  (tests to be added)
```

4. Commit History

Eight meaningful commits were made. Each message explains the change; there are no empty or repeated commits. Six are on `main` and two are on the task branch.

```
* 149f4f3 Update README with project status and link to the collaboration record
* 3a29fd7 Add team collaboration record
* 10e2de9 Add src/ and tests/ folder guides for the implementation phase
* 96a8f6b Add UML and ER diagram images to diagrams/
* b3a4185 Add Part 3 system modeling document to docs/modeling
* 3e17442 Add Part 2 Software Requirements Specification to docs/requirements
* 6b5a81a Add Part 1 project proposal to docs/proposal
* 0aa294e Initialize repository with README and .gitignore
```

Commit messages (git log, with dates)

```
0aa294e  2026-10-19  Initialize repository with README and .gitignore
6b5a81a  2026-10-19  Add Part 1 project proposal to docs/proposal
3e17442  2026-10-19  Add Part 2 Software Requirements Specification to docs/requirements
b3a4185  2026-10-19  Add Part 3 system modeling document to docs/modeling
96a8f6b  2026-10-19  Add UML and ER diagram images to diagrams/
10e2de9  2026-10-19  Add src/ and tests/ folder guides for the implementation phase
3a29fd7  2026-10-19  Add team collaboration record
149f4f3  2026-10-19  Update README with project status and link to the collaboration record
```

5. Task Branch and Pull Request

The branch `docs/team-collaboration-record` was created from `main` for one genuine project update: adding the team collaboration record and updating the README status. It contains two commits and is ready to be pushed and opened as a pull request for teammate review before merging.

```
git switch -c docs/team-collaboration-record
<edit files>
git status
git diff
git add docs/team/collaboration-record.md
git diff --staged
git commit -m "Add team collaboration record"
git push -u origin docs/team-collaboration-record
```

Pull request description

```
Title: Add team collaboration record and update README status

Purpose: Document how Group 8 uses Git (roles, branch/commit workflow,
  review rule, conflict handling), as required by Part 4.
Changes:
  - Add docs/team/collaboration-record.md
  - Update README.md: Part 4 status and link to the record
Checks: Opened and read the record; no secrets or personal data included;
  consistent with Part 1 member responsibilities.
Known limitations: None. Reviewer: one teammate (not the author) approves
  before merge into main.
```

Review evidence: screenshot of the pull request and the teammate's approval (added after the PR is opened on GitHub).

Team Collaboration Record - Group 8

Project: Second-hand Item Exchange System | **Supervisor:** Dr. Sirwan | **Platform:** GitHub

1. Member responsibilities

- **Lizaa Chato** - Team coordinator and repository owner: README, requirements, documentation, final report, and merging approved pull requests.
- **Bushra Faris** - System analysis and UML modelling: `diagrams/` and `docs/modeling/`.
- **Passar Khdir** - Database design and back-end development: `src/backend/`, `src/database/`.
- **Ariyan Walid** - User interface, front-end development, and testing: `src/frontend/`, `tests/`.

Each member mainly works in the folders above; anyone may propose a change to any folder through a pull request.

2. Branch and commit workflow

1. `main` always contains reviewed, working work. Nobody pushes directly to `main`.
2. Before starting, run `git switch main` and `git pull`, then create a task branch: `git switch -c /`.
   Types: `feature/` (new function), `docs/` (documents), `fix/` (bug fix). Example: `feature/item-list`.
3. Work in small, focused commits. Before each commit run `git status` and `git diff --staged`.
4. Commit messages say what changed and why, in the imperative form: "Add class diagram", "Fix login validation". Never "update" or "changes".
5. Push the branch (`git push -u origin `) and open a pull request that describes the purpose, the changes, the tests done, and known limitations.
6. After approval, the PR is merged into `main` and the task branch is deleted. Force-push is never used on shared branches.

3. Review and testing rule

- Every pull request needs **at least one approval from a teammate who is not the author** before it is merged.
- The reviewer checks that: the purpose is clear; files are in the correct folder; the change matches the SRS and the diagrams (FR/NFR IDs); no passwords, tokens, or private data are included; and, for code, the tests pass.
- Code changes must include or update a test. Documents are opened and read before approval.
- Review comments are answered within two days; the author fixes them in new commits on the same branch.

4. Conflict handling

- Run `git pull` on `main` often, and again before opening a PR, to find conflicts early.
- If Git reports a merge conflict: (1) inspect both versions; (2) talk with the teammate who made the other change and agree on the intended result, checking the SRS if it is a requirement question; (3) edit the file and remove the conflict markers; (4) run the tests or open the document; (5) `git add`, then commit the merge.
- Never choose one side blindly. If the team cannot agree, the coordinator decides, and for requirement questions Dr. Sirwan is asked.
- Two people should not edit the same file at the same time; tasks are assigned in the group chat first.

5. Safety rules

- Never commit passwords, tokens, private keys, or personal data. Secrets stay in a local `.env` file, which is ignored by `.gitignore`.
- If a secret is committed by mistake, tell the team immediately and change (rotate) that secret; deleting the file later is not enough.